FASE 2 — ORQUESTACIÓN MULTIAGENTE Y STORYTELLING

Transición hacia una arquitectura estructurada con CrewAI, Ollama y Qwen2.5

Objetivo: Implementar un pipeline orquestado	**Fecha Objetivo:** Octubre de 2026
Entregable: Informe técnico, arquitectura y código base	**Estado:** En desarrollo / Verificado
Nombre: Oscar Manuel Contreras Gacha	**Código:** 20221020052

1. Transición de la Fase 1 a la Fase 2

Durante la Fase 1 de este proyecto de investigación, se experimentó mediante el uso directo e independiente de distintas interfaces de modelos de lenguaje basados en la nube (tales como Gemini, ChatGPT, Claude y DeepSeek). Si bien este enfoque exploratorio permitió evaluar la capacidad creativa y literaria de diversos LLM y seleccionar el estilo narrativo adecuado, dejó al descubierto severas limitaciones metodológicas y técnicas al momento de escalar el proceso hacia la producción de contenidos multimedia estructurados.

1.1. Limitaciones del Enfoque Manual y No Orquestado

- **Transferencia manual de información:** La ausencia de un canal automatizado obligaba a copiar y pegar resultados intermedios entre diferentes interfaces de usuario (por ejemplo, trasladar una escaleta de texto plano al prompt de un generador de imágenes o de audio).
- **Pérdida y degradación del contexto narrativo:** Al interactuar de forma desarticulada, los modelos perdían las restricciones canónicas sobre los arcos dramáticos, provocando desviaciones en la trama central.
- **Inconsistencias en entidades y escenarios:** Se identificaron contradicciones frecuentes en las características físicas de los personajes, la indumentaria, los nombres y la topografía de los escenarios entre una escena y otra.
- **Redundancia operativa:** Requería la repetición constante de instrucciones contextuales (preámbulos extensos o *system prompts* repetitivos) en cada etapa del flujo de trabajo.

Estas dificultades no se deben a una incapacidad intrínseca de los modelos de lenguaje modernos para procesar información, sino a la **ausencia de una arquitectura de software y un flujo de trabajo centralizado** que controle rigurosamente el paso de datos entre las diferentes etapas de la cadena de valor del storytelling.

2. Fundamentos de CrewAI en la Arquitectura del Proyecto

Para superar las barreras de la fase anterior, se adoptó **CrewAI**, un framework de código abierto diseñado para orquestar equipos de agentes autónomos basados en IA. Este entorno permite descomponer problemas narrativos complejos en tareas secuenciales bien delimitadas.

2.1. Componentes Principales

- **Agents (Agentes):** Entidades configuradas con un rol específico, un objetivo preciso y un *backstory* (trasfondo) que delimita su especialización y tono operativo (por ejemplo, el arquitecto de escaletas o el escritor creativo).

- **Tasks (Tareas):** Unidades de trabajo atómicas que especifican qué insumos procesar, qué restricciones cumplir y cuál debe ser el formato estricto del resultado esperado (*expected output*).
- **Crew (Equipo):** El contenedor lógico que agrupa a los agentes y las tareas, estableciendo el orden de ejecución y las políticas de cooperación.
- **Process (Proceso):** Mecanismo de coordinación que, en esta implementación, opera de manera secuencial (*Process.sequential*), asegurando que la salida de una tarea alimente de forma controlada a la siguiente.
- **Context y Outputs:** Canales estructurados mediante los cuales las tareas transfieren datos limpios (como cadenas en formato Markdown o JSON) entre fases consecutivas del pipeline, evitando la amnesia de contexto.

3. Mitigación de la Pérdida de Información mediante Orquestación

La arquitectura implementada en la Fase 2 ataca directamente los problemas de inconsistencia de la Fase 1 a través de decisiones de diseño de software bien definidas:

1. **Delimitación estricta de fases:** El flujo se divide en etapas independientes (Estructuración de escaleta → Redacción de prosa → Generación de Storyboard → Síntesis de Audio), evitando que un agente intente realizar múltiples tareas contradictorias simultáneamente.
2. **Especialización de roles por agente:** Cada agente utiliza un prompt de sistema optimizado para su función específica, reduciendo el ruido cognitivo del modelo.
3. **Inyección explícita de metadatos canónicos:** El sistema inyecta un diccionario centralizado de metadatos (personajes, espacio, tiempo y trama) en el punto de partida del flujo, asegurando que todos los agentes operen bajo la misma fuente de verdad.
4. **Validación y control iterativo:** Se implementaron bucles de supervisión en puntos críticos del pipeline para revisar los entregables antes de propagarlos a las siguientes etapas multimedia.

Es importante destacar que CrewAI no garantiza por sí mismo la persistencia absoluta del contexto; la calidad de la salida depende críticamente de cómo se estructuran las tareas, cómo se limpian los formatos de transferencia y cómo se restringen las respuestas de los modelos locales.

4. Entorno de Ejecución: Ollama y Qwen2.5

Como motor de inferencia local se utiliza **Ollama**, el cual permite hospedar y ejecutar modelos de lenguaje de código abierto directamente en la infraestructura del investigador, garantizando privacidad, control de parámetros y cero costos por llamada a API externa.

Como modelo subyacente se ha seleccionado **Qwen2.5** (en su variante optimizada para procesamiento local). A diferencia de modelos ligeros evaluados previamente (como Llama 3.2 de menor escala, que presentó problemas de alucinación y pérdida de restricciones complejas ante metadatos extensos), Qwen2.5 demostró una excelente capacidad de razonamiento lógico, comprensión profunda de la gramática en español y alta fidelidad en el seguimiento de instrucciones estructuradas tanto para la fase de estructuración (JSON/Markdown) como para la redacción de prosa literaria fluida y la generación de prompts en inglés para storyboard.

5. Arquitectura y Flujo de Ejecución del Sistema

El script orquestador (‘main.py‘) coordina la ejecución secuencial del sistema. A continuación se detalla fragmento clave de la implementación en código fuente utilizada para la generación automatizada de los capítulos:

```
# Instanciaci n de agentes con el modelo Qwen2.5 local v a Ollama
llm_local = LLM(model="ollama/qwen2.5", base_url="http://localhost:11434")

guionista = Agent(
```

```
    role=config_agentes['guionista_estructurador']['role'],
    goal=config_agentes['guionista_estructurador']['goal'],
    backstory=config_agentes['guionista_estructurador']['backstory'],
    verbose=True, allow_delegation=False, llm=llm_local
)

escritor = Agent(
    role=config_agentes['escritor_creativo_capitulos']['role'],
    goal=config_agentes['escritor_creativo_capitulos']['goal'],
    backstory=config_agentes['escritor_creativo_capitulos']['backstory'],
    verbose=True, allow_delegation=False, llm=llm_local
)

# Ejecuci n secuencial controlada por CrewAI
equipo_escaleta = Crew(agents=[guionista], tasks=[tarea_estructurar], process=Process.
    sequential)
escaleta_resultado = equipo_escaleta.kickoff().raw
```

Listing 1: Extracto del orquestador principal basado en CrewAI.

5.1. Pipeline Multimedia Integrado

Una vez que el agente escritor genera el archivo de texto en prosa (‘.md‘), el sistema ejecuta automáticamente dos procesos complementarios:

1. **Módulo TTS (Edge-TTS):** Limpia la sintaxis Markdown mediante expresiones regulares y utiliza la voz neuronal en español (‘es-MX-JorgeNeural‘) para generar de forma asíncrona el archivo de audio hablado (‘.mp3‘).
2. **Agente Director de Storyboard:** Analiza la prosa del capítulo y desglosa entre 5 y 10 escenas clave en formato JSON estructurado, incluyendo descripciones visuales y prompts optimizados en inglés para futuros motores de generación de imágenes.

6. Ventajas, Limitaciones y Trabajo Futuro

6.1. Ventajas Potenciales Frente a la Fase 1

* Automatización de la transferencia de datos entre agentes, eliminando la fricción del copiado manual. * Coherencia estructural y temática sostenida gracias a la inyección centralizada de metadatos. * Reproducibilidad del pipeline narrativo y multimedia bajo un entorno local controlado.

6.2. Limitaciones Actuales

* Dependencia de la ventana de contexto y los sesgos del modelo local seleccionado. * Necesidad de un diseño riguroso en las descripciones de las tareas para evitar desviaciones estilísticas. * La generación automática de video e imágenes finales se mantiene en fase conceptual o de evaluación preliminar debido a la complejidad de cómputo en tiempo real.

6.3. Trabajo Futuro

Mejorar el flujo de trabajo, describcion de roles, y enfoques de tareas para una generalización de trabajo independiente del tipo de trama que se especifique. * Integración de herramientas automatizadas para la generación directa de imágenes mediante Stable Diffusion / ComfyUI conectadas al JSON del storyboard